

GALILEO Group 31 october 2025 Deadline Report

Francesco Pivotto 2158296

Giorgia Amato 2159999

Alessio Demo 2142885

13 novembre 2025

Indice

1	Introduction	2
2	Core Architectural Design and Execution Flow:	2
3	Baseline NL	3
3.1	General Procedure	3
3.2	Dataset Categorization and Contextualization	3
3.3	Architecture:	4
4	Baseline SQL	5
4.1	General Procedure	5
4.2	Architecture and Implementation	5
4.3	Command-line Interface and Logical Flow	6
4.4	Results and Evaluation	6
5	Baseline Palimpzest	7
5.1	Overall Architecture	8
5.2	Initialization (__init__)	8
5.3	Loading and Indexing (Load_and_index)	8
5.4	Retrieval of Relevant Chunks (retrieve)	8
5.5	Saving and Loading Index	8
5.6	Results and Evaluation	9
6	Contributions	9

1 Introduction

In the previous project task we assessed the query execution on the database for each dataset and after that, we use `galois_eval.py` for the evaluation process on the results we obtained from our implementation. Now in this task we will describe the development process implemented for replicate the following baselines using the Internal Knowledge datasets:

- **SQL baseline:** The system prompt the LLM model directly with a SQL query.
- **NL baseline:** The system prompt the LLM model directly with a NL prompt.
- **Palimpzest baseline:** The system evaluates the pure reasoning and synthesis capabilities of the LLM when operating under a Retrieval-Augmented Generation (RAG) paradigm (**BONUS**). Notice that the current Palimpzest baseline is a simple RAG setup right now. It needs more work because it doesn't exactly match the advanced data handling and reasoning ideas from the original Galois paper."

For all these baselines we will show also the tables that reports the evaluation measures and metrics. After this brief introduction, let's take a look at all the details of the various baselines.

2 Core Architectural Design and Execution Flow:

The central logic of the codebase is built following the logic of some software design patterns, ensuring modularity and maintainability:

- **Adapter** : This is implemented in `llm_wrappers.py`. It standardizes the interface for heterogeneous LLM providers (e.g., Gemini, Watsonx), enabling the core execution engine to interact with them through a uniform API.
- **Factory** : This pattern is located in `llm_factory.py`. It decouples client components (e.g., `sql_baseline.py`, `nl_baseline.py`) from the specific instantiation logic of concrete LLM wrapper objects. Extensibility is achieved simply by updating the `PROVIDER_MAP`.
- **Template** : This is defined by `LLMBaseWrapper` in `llm_wrappers.py`. It establishes the invariant sequence for retrieving the LLM instance, while deferring the implementation of the provider-specific connection details (`create_llm_instance`) to concrete subclasses.
- **Pipeline (LCEL):** This pattern is realized in `build_lcel_chain.py` within `baseline_tools.py`. It establishes a structured, sequential execution flow: Context Generation → Prompt Construction → LLM Invocation.

Standardized Execution Workflow:

The first step is the `get_db_context` function dynamically prepares the necessary input context such as the database schema or simulated RAG data for the Palimpzest baseline based on the requested baseline type, next the derived context is injected into a **ChatPromptTemplate**.

This stage manages the sophisticated prompting logic, including the insertion of system

instructions and the required JSON output formatting instructions via **PydanticOutputParser**. After that The fully prepared prompt is transmitted to the configured LLM instance and finally the `parse_llm_response` function standardizes the output, parsing the required JSON structure and extracting critical metadata, including execution time and estimated token usage.

3 Baseline NL

The Baseline NL module provides a reference framework for evaluating Large Language Models (LLMs) through direct interaction via natural language prompts. This approach assesses the model’s intrinsic reasoning and knowledge by prompting it with natural language (NL) questions, optionally enriched with structural and contextual information derived from a database schema or a specific query results. All outputs are standardized in a unified FULL JSON format to facilitate downstream evaluation.

3.1 General Procedure

1. **Prompt Loading:** Natural language prompts are loaded from dataset-specific resources (`queries_<dataset-name>.txt`). Each prompt corresponds to a query from the same dataset.
2. **LLM interaction:** Prompts are processed through a **LangChain-based pipeline**, which integrates:
 - The general task definition and LLM role specification.
 - The corresponding database schema, extracted via DuckDB.
 - Optional contextual data (e.g. query results for Model Context datasets).
 - The natural language question itself
3. **Response Parsing and Storage:** The model’s output is parsed using a **PydanticOutputParser**, ensuring compliance with the required FULL JSON format. Each response is stored in an individual JSON file, forming a consistent source for evaluation.

3.2 Dataset Categorization and Contextualization

1. **Internal knowledge:** In these datasets, the model relies exclusively on its pretrained knowledge and linguistic reasoning capabilities. The prompt includes only the natural language question and the database schema for both SQL and N baseline, but no factual data retrieved from the database.
2. **Model Context:** For what concerns this project task we did not use Model Context datasets in either NL or SQL baseline. These kind of datasets are used in the palimpsest baseline described in the next section, but it still needs to be improved (in the next tasks)

One common behavior is that each response produced by any baseline is written in a JSON file that fits the following FULL JSON format:

```
{
  "result_set": [
    { "originaltitle": "The Three Musketeers" },
    { "originaltitle": "The Count of Monte Cristo" }
  ],
  "time": 1.84,
  "tokens": 312
}
```

where:

- **result_set:** Denotes the structured LLM output.
- **time:** Represents the total time taken by the LLM to generate the response.
- **tokens:** Indicates the number of tokens processed during the interaction

3.3 Architecture:

The architecture is composed of three main components:

1. Configuration Layer:

- API keys for the supported providers (**IBMWatsonx** and **Google Gemini**) are loaded from a **.env** file.
- The **Config_Loader** selects the target LLM provider, and the appropriate wrapper is instantiated through **get_llm_wrapper()**.
- Each wrapper extends the **LLMBaseWrapper** class, implementing provider-specific logic for model initialization, token counting, and response handling.

2. Chain Construction:

The **build_lcel_chain()** method constructs a **LangChain Expression Language (LCEL)** pipeline comprising:

- **Database Context Retrieval:** **get_db_context()** extracts schema information and, for Model Context datasets, executes the original SQL query using DuckDB database instance of corresponding dataset.
- **Prompt Composition:** **ChatPromptTemplate** merges two layers, a **SYSTEM_PROMPT** defining the LLM's role and behavior, and a **HUMAN_PROMPT** containing the NL question and formatting directives.
- **Structured Output Parsing:** The **PydanticOutputParser** enforces adherence to the predefined FULL JSON schema.

3. Execution Flow

- For each dataset, SQL queries and corresponding NL prompts are loaded.
- The (**prompt**, **query**) pairs are processed through **chain.invoke()**.
- Responses are parsed, validated, and augmented with the metadata such as execution time and token usage.
- The resulting files are serialized via **save_baseline_to_json()** for evaluation purposes.

4 Baseline SQL

The Baseline SQL module extends the evaluation framework to the case where the model is queried through SQL statements instead of natural language questions. The goal is to analyse how the different LLM providers are able to interpret SQL queries, reason over the underlying database schema and data, and produce a structured JSON answer that can be evaluated with the same metrics used for the NL baseline.

4.1 General Procedure

The Baseline SQL pipeline follows a procedure that is conceptually similar to the one adopted for the NL baseline:

1. **Query loading.** For each dataset all SQL statements are loaded from the corresponding `queries_<dataset>.sql` file by means of the helper function `load_queries_from_folder()`.
2. **Schema and context extraction.** The DuckDB database associated with the dataset is opened and its schema is extracted. As for the NL baseline, datasets are divided into two categories:
 - *Internal Knowledge (IK)*: only the database schema is provided to the model as external context;
 - *Model Context (MC)*: the SQL query is also executed on DuckDB and the tuples returned by the query are supplied to the model as additional context.
3. **Prompt construction.** A structured prompt is built that combines a high level description of the task (SQL interpretation and result formatting), the schema information and, for MC datasets, a textual representation of the rows returned by the query, together with the SQL statement itself. The helper `build_sql_prompt()` wraps the query into a concise instruction.
4. **LLM invocation and parsing.** The prompt is sent to the selected LLM provider (Google Gemini or IBM watsonx). The answer is parsed through a `PydanticOutputParser` that enforces a common FULL JSON schema including the fields `result_set`, `time` and `tokens`. In case of parsing errors, the raw content is logged and an empty `result_set` is returned together with timing information.
5. **Result serialisation.** For each query a JSON file is produced and saved in a provider-specific directory inside the results folder. These files are then used by the evaluation scripts to compute the same metrics already introduced for the NL baseline.

4.2 Architecture and Implementation

The implementation of the Baseline SQL module is split into two main components, one for each provider.

For **Google Gemini** the core logic is implemented in the `baseline_sql_gemini.py` module. The main responsibilities of this component are:

- opening the DuckDB database of the selected dataset and extracting the schema;

- executing, in the MC case, the SQL query to obtain the raw tuples;
- building the prompt by combining schema, optional raw data and the SQL statement;
- invoking the model and parsing the output with a Pydantic schema (**Response**) that contains the **result_set** and metadata such as execution time and token usage;
- writing one JSON file per query in a deterministic path that depends on the provider and the dataset.

The function `run_sql_baseline_gemini()` orchestrates these steps for one or more datasets. If no datasets are specified on the command line, the selection is read from the YAML configuration (field `database.run`).

For **IBM watsonx** the Baseline SQL follows the same high level structure. The helper `run_sql_baseline_watsonx()` loads the queries, distinguishes between IK and MC datasets, executes the appropriate baseline function and finally serialises the results using the same FULL JSON format. In this way the two providers can be compared directly on the same evaluation metrics.

4.3 Command-line Interface and Logical Flow

The unified command-line interface is implemented in `src/main.py` and controls the execution of both NL and SQL baselines. The script exposes:

- a positional argument **datasets**, that can contain zero, one or multiple dataset names (for example `WORLD`, `GEO`, `MOVIES`);
- the option `-mode`, with possible values `nl`, `sql` or `both`, which selects whether to run only the NL baseline, only the SQL baseline or both;
- the option `-provider`, with possible values `gemini` and `watsonx`, that can override the provider defined in the YAML configuration file.

At a high level, the logical flow is the following:

1. parse the command-line arguments;
2. determine the list of datasets, either from the CLI or from the configuration;
3. according to the selected mode, invoke the NL baseline, the SQL baseline, or both;
4. for each baseline, dispatch to the provider-specific implementation (Gemini or watsonx) and store the resulting JSON files.

4.4 Results and Evaluation

The quantitative results of the Baseline SQL module are reported together with those of the NL baseline in the tables that summarise, for each provider, the F1-CELL, CARDINALITY and TUPLE-CONSTRAINT metrics, their average score, the inference time and the number of tokens produced. The results obtained from the Baseline NL and SQL baseline module were evaluated using standard metrics to assess the performance

of the LLMs in both Natural Language (NL) and SQL contexts. The model used for the **gemini** provider is: **gemini-2.5-flash**, instead for **IBMWatsonx** provider we used **meta-llama/llama-3-3-70b-instruct**. All systems are tested on the datasets described in Table 2 of the previous project task, excluding Premier and Fortune like the galois authors did.

Metric	NL	SQL
F1-CELL	0.391	0.537
CARDINALITY	0.644	0.733
TUPLE-CONSTR.	0.182	0.331
AVG-SCORE	0.406	0.534
#TOKENS(M)	39965	126946
AVG-TIME	4.812	7.561

Tabella 1: GEMINI results

Metric	NL	SQL
F1-CELL	0.436	0.523
CARDINALITY	0.701	0.718
TUPLE-CONSTR.	0.212	0.243
AVG-SCORE	0.450	0.495
#TOKENS(M)	15165	11606
AVG-TIME	5.353	4.485

Tabella 2: WATSONX results

5 Baseline Palimpzest

The baseline aims to automatically retrieve relevant text segments (from Markdown or plain text files) to construct additional contextual information for a Retrieval-Augmented Generation (RAG) pipeline. The file resources that we used for this baseline was retrieved from the GALOIS repository within the `core/src/test/resources/rag-fortune` and `core/src/test/resources/rag-premier` folders. In short, it builds a semantic knowledge base from textual documents and indexes them using embeddings and FAISS.

Talking about FAISS, it is an open-source library developed by Facebook AI Research that provides efficient similarity search and clustering of dense vectors.

The reason why we use this library is that we need something that allows us to perform fast vector indexing and similarity searches over large datasets of text embeddings, in a reasonable time and CPU usage.

For what concerns the resources usage (CPU, GPU) we used only the CPU for ensuring portability and easy baseline testing without GPU support. Actually the most of the times, the baseline was tested on a machine with a very low GPU power (a Laptop with a integrated GPU, not a dedicated one); furthermore when we try to run the baseline using also the GPU the running time needed to download the dedicated Python Library for GPU support was much higher than the time needed to download the CPU ones, so finally we decided only to use the latter. For the same time saving reasons we used

a light sentence transformer model (**sentence-transformers/all-MiniLM-L6-v2**) for the sentence embeddings generation, in fact it has a good trade-off between the running time speed and the semantic precision and also avoids the overhead of large transformer models while still maintaining strong semantics representations.

Since our implementation needs to be easily deployable and testable on different machines, we decided to avoid heavy models that require high computational resources.

5.1 Overall Architecture

Generally the baseline consists of three main components:

- **MarkdownRAGBackend class:** It handles file loading, chunking, embedding and FAISS indexing.
- **pz_context():** main method that uses the backend to build contextual input (relevant chunks) for the Palimpzest model.
- **Backend:** Represents all the "behind the scenes" logic.

5.2 Initialization (`__init__`)

Here a HuggingFaceEmbeddings instance is created to generate text embeddings using a specified model, the chosen model is **sentence-transformers/all-MiniLM-L6-v2** as already seen.

5.3 Loading and Indexing (`Load_and_index`)

This method performs the full data preparation and indexing pipeline:

- Searching for a .md, .txt or extensionless files
- Converts each file into LangChain documents, preserving metadata.
- The text is divided in chunks (128 tokens per chunk for premier and 400 tokens per chunk for fortune) balancing semantic coherence and retrieval precision, and also preserves contextual continuity across chunks setting an overlap of 20 tokens.
- Creation of FAISS index from the generated embeddings for efficient similarity search.

5.4 Retrieval of Relevant Chunks (`retrieve`)

Essentially the method returns the **top_k** most semantically similar text chunks for a given query, so it performs the semantic retrieval.

5.5 Saving and Loading Index

Finally the FAISS index is saved in a persistent way to avoid re-indexing on subsequent runs, and it can be loaded back when needed.

We would like to emphasize that the reason we used the text files available in the GALOIS repository as resources, rather than, for instance, a subset of records from the

Fortune or Premier datasets (or similar sources), is that the original idea behind the Palimpzest baseline (as stated in the GALOIS paper) is to evaluate the LLM’s capabilities in a RAG paradigm using external textual resources that are not part of the datasets themselves.

5.6 Results and Evaluation

Metric	NL	SQL
AVG-SCORE	0.393	X
#TOKENS(M)	2390	X

Tabella 3: PALIMPZEST results

6 Contributions

- **Alessio Demo (Badge number: 2142885)** First Project Task: Developed function database connection and instance creation of DuckDB database manager for each dataset using the ingest files (.duckdb files). Developed function for retrieving the SQL queries from .sql files, processing the queries and executing them on DuckDB database instance and finally store it in a json file. Second Task: Developed the Baseline NL function for interacting with the LLM.
"Early-stage" Palimpzest baseline implementation.
- **Francesco Pivotto (Badge number: ...)**
- **Giorgia Amato (Badge number: ...)**